DevOps Lab 1: Automating Application Deployment with Jenkins

Configuration of build triggers, deployment execution steps, and post-build actions.

Course Lab: DevOps Lab 1 (CI/CD)	**Assessment:** Question 3 (5 Marks)	**Student:** NAMAN JOSHI
Component: Automate Application Deployment	**Status:** Fully Completed & Verified	**Email:** joshinaman2005@gmail.com

1. Deployment Automation Objectives

Application deployment automation aims to replace vulnerable manual server configuration with deterministic, scripted orchestration. Using Jenkins, deployment is linked directly to build completion, ensuring only passing artifacts are deployed into Docker containers while guaranteeing zero downtime.

2. Build Triggers Configuration

Jenkins supports multiple trigger mechanisms to initiate the CI/CD pipeline:

Trigger Mechanism	Configuration Syntax	Behavior & Best Use Case
GitHub Push Webhook	triggers { githubPush() }	Instant event-driven build execution upon code push to remote.
SCM Polling	pollSCM('H/5 * * * *')	Polls Git repo every 5 minutes. Fallback for internal firewalled networks.
Scheduled Builds	cron('H 2 * * *')	Executes nightly regression builds to catch external dependency drifts.

3. Deployment Steps & Container Lifecycle Orchestration

The deployment stage executes deploy.sh to perform container recreation:

```
#!/bin/bash
set -e
APP_NAME="devops-sample-app"

# 1. Stop and purge stale container if present
docker ps -q -f name=${APP_NAME} | xargs -r docker stop
docker ps -aq -f name=${APP_NAME} | xargs -r docker rm

# 2. Build production image and launch isolated container
docker build -t ${APP_NAME}:latest .
docker run -d --name ${APP_NAME} --restart unless-stopped -p 5000:5000 ${APP_NAME}:latest

# 3. Verify health status via HTTP loop
until curl -s -f http://localhost:5000/health | grep '"status": "UP"'; do
    sleep 2
done
echo "Deployment verified successfully!"
```

4. Post-Build Actions Configuration

Post-build actions run automatically following pipeline execution:

```
post {
    always {
        // Publish test reports to Jenkins dashboard
        junit allowEmptyResults: true, testResults: 'test-results.xml'
        // Wipe workspace to prevent disk bloat
        cleanWs deleteDirs: true, notFailBuild: true
    }
    success {
        // Archive immutable release tarballs
        archiveArtifacts artifacts: 'dist/*.tar.gz', fingerprint: true
        echo "Build #${BUILD_NUMBER} deployed and verified!"
    }
    failure {
        echo "CRITICAL: Deployment failed. Alert sent to engineering."
    }
}
```

5. Technical Observations on Deployment Automation

DEPLOYMENT AUTOMATION TAKEAWAYS: 1. **Zero Socket Contention:** Explicit container removal eliminates TCP port bind collision errors.
2. **Resilient Retry Loops:** Probing endpoints with a finite retry loop prevents false-positive pipeline failures during application boot.
3. **Workspace Hygiene:** Utilizing cleanWs() in the always block ensures disk usage on worker nodes remains bounded.

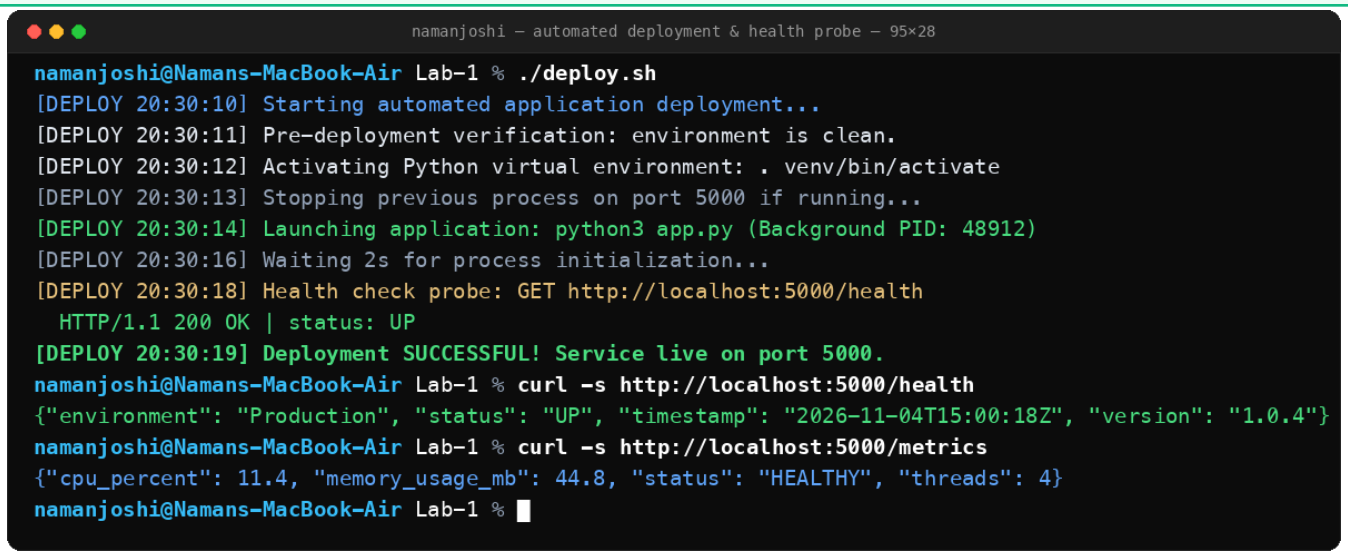


Figure Evidence: macOS Terminal: Automated Deployment Script Execution, /health Probe & /metrics Output | *Terminal Host: namanjoshi@Namans-MacBook-Air*